Exercises

Exercise 5: Visualising data using base R

Read Chapter 4 to help you complete the questions in this exercise.

1. As in previous exercises, either create a new R script or continue with your previous R script in your RStudio Project. Again, make sure you include any metadata you feel is appropriate (title, description of task, date of creation etc) and don't forget to comment out your metadata with a `#` at the beginning of the line.

2. You do not need to download anything for this exercise. You already have *'cardiacdata.txt'* in the `data` directory of your RStudio Project, from Exercise 3. If for some reason you haven't, go back to the **Data** link, download *'cardiacdata.xlsx'* and save it as a tab delimited file called *'cardiacdata.txt'* as you did before.

3. A quick reminder of what these data are. They come from a cohort study of the risk factors for cardiovascular disease, in which 163 adults aged between 55 and 75 were examined and a range of measurements taken: age and sex, systolic and diastolic blood pressure (mmHg), body mass index (kg/m^2), total cholesterol, HDL cholesterol and triglyceride (all mmol/l), units of alcohol drunk in the previous week, and smoking status. Note that you are importing the original file again, not the cleaned version you exported at the end of Exercise 4. Starting from the raw data every time, and doing your cleaning in a script, is what makes an analysis reproducible.

4. Import the *'cardiacdata.txt'* file into R using the `read.table()` function and assign it to a variable named `cardiac`. Use the `str()` function to display the structure of the dataset and the `summary()` function to summarise it. As you found in Exercise 3, the `sex` and `smoking` variables are coded as integers, but they are really categories. Create a new variable in the `cardiac` dataframe for each of them, recoded as a factor with meaningful labels (`sex` is 1 for Female and 2 for Male; `smoking` is 1 for Current, 2 for Ex and 3 for Never). Use the `str()` function again to check the coding of these new variables. Almost every plotting function in this exercise treats a factor differently from a number, so this step is not optional housekeeping - get it wrong and your boxplots will come out as nonsense.

5. How many patients are there in each combination of smoking status and sex (hint: remember the `table()` function?)? Don't forget to use the factor recoded versions of these variables. Is the split between the smoking categories similar for women and men? Are any of the combinations small enough to worry about if you wanted to compare them?

6. The humble cleveland dotplot is a great way of identifying if you have potential outliers in continuous variables (see Section 4.2.4). Create dotplots (using the `dotchart()` function) for the following variables; `bmi`, `systolic`, `tchol` and `alcohol`. Do these variables contain any unusually large or small observations? Don't forget, if you prefer to create a single figure with all 4 plots you can always split your plotting device into 2 rows and 2 columns (see Section 4.4 of the book). Use the `pdf()` function to save a pdf version of your plot(s) in the `output` directory you created in Exercise 1 (see Section 4.5 of the book to see how the `pdf()` function works).

7. You met that extreme `bmi` value in Exercise 4, when you checked the ranges with `summary()` and found a maximum of 514.60. A body mass index of 514 is not possible: it is 51.46 with the decimal point in the wrong place. What the dotplot adds is not the discovery, but the damage: one bad value has made the plot useless for looking at the other 162 patients. Use the `which()` function to identify which observation it is, confirm the value with the square bracket `[ ]` notation, set it to `NA` as you did in Exercise 4, and then redo the dotchart. Now look again at all four plots. Several points still sit well away from the rest. Which ones, and what should you do about them?

8. When exploring your data it is often useful to visualise the distribution of continuous variables. Take a look at Section 4.2.2 and then create histograms for the variables; `bmi`, `systolic`, `tchol` and `alcohol`. Again, it's up to you if you want to plot all 4 plots separately or in the same figure. Export your plot(s) as pdf file(s) to the `output` directory. One potential problem with histograms is that the distribution of data can look quite different depending on the number of 'breaks' used. The `hist()` function does its best to create appropriate 'breaks' for your plots (it uses the Sturges algorithm for those that want to know) but experiment with changing the number of breaks for the `systolic` variable to see how the shape of the distribution changes (see Section 4.2.2 of the book for further details of how to change the breaks).

9. Scatterplots are great for visualising relationships between two continuous variables (Section 4.2.1). Plot the relationship between `systolic` on the x axis and `diastolic` on the y axis. How would you describe this relationship? Now plot `hdlchol` on the x axis against `triglyceride` on the y axis. Which way does that one go, and does that make clinical sense to you? Finally, take a look at `alcohol`. It is badly skewed, and skew is a nuisance in a plot because most of the points end up squashed into a corner. One approach is to apply a transformation. Create new variables in the `cardiac` dataframe holding the square root (`sqrt()`) and natural log (`log()`) of `alcohol` and plot histograms of each. One of the two transformations goes badly wrong. Which one, and why? (hint: what is the smallest value of `alcohol`, and what is the log of it?) Save your plots as a pdf file, or try jpeg or png format using the `jpeg()` or `png()` functions (Section 4.5) if you feel the need for a change.

10. When visualising differences in a continuous variable between levels of a factor (categorical variable) then a boxplot is your friend (avoid using bar plots - Google 'bar plots are evil' for more info). Create a boxplot to visualise the differences in total cholesterol at each level of smoking status (don't forget to use the recoded version of this variable you created in Q4). Include x and y axes labels in your plot. Make sure you understand the anatomy of a boxplot before moving on - please ask if you're not sure (also see Section 4.2.3 of the book). An alternative to the boxplot is the violin plot. A violin plot is a combination of a boxplot and a kernel density plot and is great at visualising the distribution of a variable. To create a violin plot you will first need to install the `vioplot` package from CRAN and make it available using `library(vioplot)`. You can now use the `vioplot()` function in pretty much the same way as you created your boxplot (again Section 4.2.3 of the book walks you through this).

11. To explore the relationships between multiple continuous variables it's hard to beat a pairs plot. Create a pairs plot for the variables; `age`, `systolic`, `diastolic`, `tchol`, `hdlchol`, `triglyceride` and `bmi` (see Section

4.2.5 of the book for more details). If it looks a little cramped in RStudio then click on the 'zoom' button in the plot viewer to see a larger version. One of the great things about the `pairs()` function is that you can customise what goes into each panel. Modify your pairs plot to include a histogram of the variables on the diagonal panel and include a correlation coefficient for each relationship on the upper triangle panels. Also include a smoother (wiggly line) in the lower triangle panels to help visualise these relationships. Take a look at the Introduction to R book to see how to do all this (or `?pairs`). Which pair of variables is most strongly related, and which variable is related to almost nothing?

End of Exercise 5